Lezione 7: Induzione, semplificazione e verifica del software

Stefano M. Nicoletti

1 Lezione 7: Induzione, semplificazione e verifica del software

1.1 Scopo della lezione

Riprendiamo la prima dimostrazione per induzione della Lezione 6. Dimostriamo altre proprietà della somma, introduciamo `simp` e combiniamo induzione, riscrittura e semplificazione. Applichiamo poi gli stessi strumenti alla verifica di un algoritmo ricorsivo sulle liste.

1.2 Definizioni e risultati disponibili

Continuiamo a usare i naturali, la somma e il doppio definiti nella Lezione 6:

`Naturale` ha i costruttori `zero` e `succ`. La funzione `somma (m n : Naturale)` ricorre sul secondo argomento, mentre `doppio (n : Naturale)` ricorre sull'unico argomento.

Ricordare quale argomento viene analizzato non è un dettaglio puramente sintattico. La definizione di `somma` può scegliere un ramo del `match` quando conosce la forma del secondo argomento. Per questo `somma m zero` e `somma m (succ n)` si riducono direttamente, mentre `somma zero n` richiede un ragionamento per induzione quando `n` è ancora una variabile.

Sono già disponibili le equazioni computazionali e il risultato conclusivo della Lezione 6:

```
open Naturale

theorem lecture07_somma_zero_destra (m : Naturale) :
    somma m zero = m := by
  rfl
```

```
open Naturale

theorem lecture07_somma_succ_destra (m n : Naturale) :
    somma m (succ n) = succ (somma m n) := by
  rfl
```

```
open Naturale

theorem lecture07_doppio_succ (n : Naturale) :
    doppio (succ n) = succ (succ (doppio n)) := by
  rfl
```

```
open Naturale

theorem lecture07_zero_somma (n : Naturale) :
    somma zero n = n := by
  induction n with
  | zero =>
      rfl
  | succ n ih =>
      unfold somma
      rewrite [ih]
      rfl
```

I primi tre risultati descrivono direttamente il comportamento delle definizioni. L'ultimo, invece, è un teorema generale ottenuto per induzione. Questa distinzione sarà utile durante tutta la lezione: alcune trasformazioni sono computazioni, altre richiedono risultati già dimostrati.

1.3 Una seconda dimostrazione per induzione

Vogliamo ora dimostrare che un successore nel primo argomento può essere portato davanti alla somma:

`somma (succ m) n = succ (somma m n).`

La forma dell'enunciato può suggerire di fare induzione su `m`, perché il successore compare accanto a quella variabile. Dobbiamo però guardare la definizione della funzione. `somma` analizza il secondo argomento, e in entrambe le somme quel posto è occupato da `n`. È quindi `n` a impedire alla definizione di scegliere tra il caso `zero` e il caso `succ`. Facciamo induzione su `n`.

```
open Naturale

theorem lecture07_succ_somma (m n : Naturale) :
    somma (succ m) n = succ (somma m n) := by
  induction n with
  | zero =>
      rfl
  | succ n ih =>
      unfold somma
      rewrite [ih]
      rfl
```

Nel caso base sostituiamo `n` con `zero`. Entrambi i lati si riducono alla stessa espressione e `rfl` conclude. Nel passo induttivo consideriamo `succ n`. `unfold somma` apre

entrambe le somme e il goal contiene la stessa uguaglianza richiesta per `n`. L'ipotesi induttiva `ih` sostituisce quella sottoespressione, dopo di che i due lati coincidono.

La scelta della variabile d'induzione segue dunque la struttura della computazione. Cerchiamo la variabile che occupa l'argomento analizzato dalla funzione e osserviamo se, nel caso successore, l'assunzione induttiva potrà riscrivere la chiamata ricorsiva ottenuta.

1.4 Associatività, commutatività e 'simp'

L'associatività afferma che il modo di raggruppare tre addendi non modifica il risultato. Nel nostro enunciato la somma esterna a sinistra ha `c` come secondo argomento. A destra, `c` compare come secondo argomento di `somma b c`, che a sua volta determina la computazione della somma esterna. Scegliamo quindi `c` come variabile d'induzione.

```
open Naturale

theorem lecture07_somma_assoc (a b c : Naturale) :
    somma (somma a b) c = somma a (somma b c) := by
  induction c with
  | zero =>
      rfl
  | succ c ih =>
      rw [lecture07_somma_succ_destra]
      rw [lecture07_somma_succ_destra]
      rw [lecture07_somma_succ_destra]
      rw [ih]
```

Nel caso `zero`, le somme che hanno `zero` come secondo argomento si riducono e `rfl` conclude. Nel caso `succ c` eseguiamo tre riscritture con `lecture07_somma_succ_destra`. La prima apre la somma esterna a sinistra. La seconda apre `somma b (succ c)` a destra. La terza apre la nuova somma esterna prodotta sul lato destro. A quel punto resta esattamente l'uguaglianza fornita dall'ipotesi induttiva, racchiusa sotto `succ`, e `rw [ih]` conclude.

La commutatività afferma invece che scambiare i due addendi non modifica il risultato. Non segue soltanto dalla computazione, perché `somma` non tratta simmetricamente i due argomenti: analizza il secondo e lascia invariato il primo. Facciamo induzione su `b`. Nel caso base il lato sinistro si riduce per definizione, mentre sul lato destro usiamo `lecture07_zero_somma`. Nel passo successore, `lecture07_somma_succ_destra` tratta il successore a destra e `lecture07_succ_somma` tratta quello che compare a sinistra dopo lo scambio.

```
open Naturale

theorem lecture07_somma_comm (a b : Naturale) :
    somma a b = somma b a := by
  induction b with
  | zero =>
      rw [lecture07_zero_somma]
      rfl
```

```
  | succ b ih =>
      rw [lecture07_somma_succ_destra]
      rw [lecture07_succ_somma]
      rw [ih]
```

Questa dimostrazione mostra perché costruiamo una raccolta di lemmi piccoli. La definizione fornisce direttamente le equazioni sul secondo argomento. I teoremi `zero_somma` e `succ_somma` forniscono le forme corrispondenti sul primo argomento. La commutatività può quindi combinare i due punti di vista.

Quando una dimostrazione richiede molte riscritture elementari, possiamo usare il semplificatore. `simp` applica equazioni orientate verso forme più semplici e svolge anche computazioni che Lean riconosce direttamente. Non cerca una dimostrazione in modo arbitrario: usa un insieme di regole chiamato *simp-set*.

Confrontiamo tre modi di usare lo stesso risultato. `rw` richiede una precisa riscrittura scelta da noi. `simp only [lemmi]` usa soltanto i lemmi elencati, oltre alle riduzioni essenziali del semplificatore. `simp [lemma]` usa le regole standard e aggiunge `lemma` per quel comando.

```
open Naturale

theorem lecture07_zero_somma_con_rw (n : Naturale) : somma zero n = n := by
  rw [lecture07_zero_somma]
```

Nel primo teorema indichiamo esattamente la riscrittura. Nel secondo chiediamo al semplificatore di lavorare con una sola regola dichiarata. Nel terzo aggiungiamo il lemma al simp-set già disponibile per quella singola chiamata. Il risultato è lo stesso, ma cambia il grado di controllo e di automazione.

```
open Naturale

theorem lecture07_zero_somma_con_simp_only (n : Naturale) : somma zero n = n
   := by
  simp only [lecture07_zero_somma]
```

```
open Naturale

theorem lecture07_zero_somma_con_simp (n : Naturale) : somma zero n = n := by
  simp [lecture07_zero_somma]
```

```
open Naturale

section SemplificazioneLocale

attribute [local simp] lecture07_zero_somma lecture07_succ_somma

theorem lecture07_succ_somma_con_simp (m n : Naturale) :
    somma (succ m) n = succ (somma m n) := by
  simp
```

```
end SemplificazioneLocale
```

L'attributo `local simp` vale soltanto all'interno della sezione. In questo modo possiamo sperimentare con un insieme di regole senza modificare tutte le dimostrazioni successive del modulo. Nel teorema interno alla sezione basta scrivere `simp`: il semplificatore trova automaticamente `lecture07_succ_somma` nel simp-set locale.

```
open Naturale

@[simp] theorem lecture07_doppio_zero :
    doppio zero = zero := by
  rfl
```

`@[simp]` registra una regola per il resto del modulo. Lo usiamo per una regola canonica come `doppio zero = zero`. Associatività e commutatività non sono regole adatte al simp-set: cambiano soltanto il raggruppamento o l'ordine degli argomenti.

La scelta delle regole è importante. Una buona regola di semplificazione porta in modo prevedibile verso una forma più breve o canonica. Orientare automaticamente l'associatività richiederebbe scegliere sempre un particolare raggruppamento. La commutatività potrebbe invece continuare a scambiare gli argomenti senza produrre una forma evidentemente più semplice. Per questi risultati preferiamo una riscrittura esplicita nel punto in cui serve.

1.5 Una dimostrazione conclusiva

Combiniamo induzione e riscritture per dimostrare che il doppio di una somma è la somma dei doppi. La sottoespressione che resta bloccata è `somma m n`, perciò facciamo induzione su `n`.

```
open Naturale

theorem lecture07_doppio_somma (m n : Naturale) :
    doppio (somma m n) = somma (doppio m) (doppio n) := by
  induction n with
  | zero =>
      rfl
  | succ n ih =>
      rw [lecture07_somma_succ_destra]
      rw [lecture07_doppio_succ]
      rw [lecture07_doppio_succ]
      rw [lecture07_somma_succ_destra]
      rw [lecture07_somma_succ_destra]
      rw [ih]
```

Il caso base segue dalle definizioni di `somma` e `doppio`. Nel passo successore, la prima riscrittura porta il successore fuori da `somma m (succ n)`. Le due riscritture con `lecture07_doppio_succ` descrivono il doppio di un successore sui due lati. Le due

successive riscritture della somma espongono la stessa struttura di successori. Infine `ih` sostituisce il doppio della somma più piccola con la somma dei doppi.

Dopo avere verificato ogni passaggio, possiamo esprimere la stessa dimostrazione con un insieme controllato di regole di semplificazione:

```
open Naturale

theorem lecture07_doppio_somma_breve (m n : Naturale) :
    doppio (somma m n) = somma (doppio m) (doppio n) := by
  induction n with
  | zero =>
      rfl
  | succ n ih =>
      simp only [lecture07_somma_succ_destra, lecture07_doppio_succ, ih]
```

La versione abbreviata non esprime un argomento matematico diverso. Usa le stesse equazioni, ma lascia che `simp only` scelga dove applicarle. Manteniamo la lista esplicita perché, in questa fase, vogliamo sapere quali risultati rendono valida la dimostrazione.

1.6 Dall'induzione alla verifica del software

Finora abbiamo usato l'induzione per dimostrare proprietà di operazioni matematiche. Lo stesso metodo permette di ragionare sui programmi. Un programma riceve un input e produce un output. Per verificarne una proprietà, definiamo con precisione il programma, formuliamo il comportamento atteso e dimostriamo che tale comportamento vale per ogni input possibile.

Provare il programma su alcuni esempi resta utile, ma non fornisce questa garanzia generale. Un test può mostrare la presenza di un errore su un input particolare. Una dimostrazione universale copre invece tutti i valori costruibili del tipo considerato.

1.6.1 6.1 Un tipo induttivo per le liste

Definiamo una lista senza usare il tipo `List` della libreria. Il parametro `α` rappresenta il tipo degli elementi contenuti. La stessa definizione può quindi essere usata per liste di naturali, stringhe o altri oggetti.

```
inductive Lista α( : Type) where
  | vuota
  | cons (testa : α) (coda : Lista α)

open Lista
```

Una `Lista α` è costruita in due modi. `vuota` non contiene elementi. `cons x xs` contiene l'elemento `x` davanti alla lista `xs`. La coda `xs` è a sua volta una lista e costituisce il dato più piccolo sul quale possono procedere ricorsione e induzione.

Il parametro `α` rende la definizione polimorfa. Non stiamo definendo una lista separata per ogni genere di elemento. `Lista Naturale` è il tipo delle liste di naturali,

mentre `Lista String` sarebbe il tipo delle liste di stringhe. I costruttori hanno la stessa struttura in entrambi i casi.

Per esempio, possiamo costruire una lista con tre naturali:

```
def listaEsempio : Lista Naturale :=
  .cons .zero
    (.cons (.succ .zero)
      (.cons (.succ (.succ .zero)) .vuota))
```

1.6.2 6.2 Concatenazione e rovesciamento

Per rovesciare una lista ci serve collocare un elemento in fondo a un risultato già costruito. Definiamo prima la concatenazione. La funzione analizza la prima lista. Se questa è vuota restituisce la seconda. Nel caso `cons`, conserva la testa e concatena ricorsivamente la coda.

```
def concatena α{ : Type} (xs ys : Lista α) : Lista α :=
  match xs with
  | .vuota => ys
  | .cons x xs => .cons x (concatena xs ys)
```

Le parentesi graffe in `{α : Type}` rendono `α` un argomento implicito. Quando scriviamo `concatena xs ys`, Lean ricava il tipo degli elementi dai tipi di `xs` e `ys`. Non dobbiamo quindi passare manualmente `α` a ogni chiamata.

La computazione segue la prima lista. Se `xs` è `vuota`, non ci sono elementi da conservare e il risultato è `ys`. Se `xs` è `cons x xs`, il risultato mantiene `x` in testa e concatena la coda con `ys`. Per esempio, usando una notazione illustrativa con parentesi quadre:

```
concatena [zero, uno] [due]
= zero :: concatena [uno] [due]
= zero :: uno :: concatena [] [due]
= [zero, uno, due].
```

Qui `[]` rappresenta `vuota` e `x :: xs` rappresenta `cons x xs`.

L'algoritmo `rovescia` segue la stessa struttura induttiva. La lista vuota resta vuota. Nel caso `cons x xs`, rovesciamo la coda `xs` e collochiamo `x` in fondo mediante `concatena`.

```
def rovescia α{ : Type} (xs : Lista α) : Lista α :=
  match xs with
  | .vuota => .vuota
  | .cons x xs => concatena (rovescia xs) (.cons x .vuota)
```

Anche `rovescia` è polimorfa. Nel caso `cons x xs`, la chiamata ricorsiva rovescia prima la coda. La lista con il solo elemento `x` viene poi concatenata in fondo. Per la lista `[zero, uno]` otteniamo:

```
rovescia [zero, uno]
= concatena (rovescia [uno]) [zero]
```

```
= concatena [uno] [zero]
= [uno, zero].
```

La chiamata ricorsiva avviene sulla coda, che contiene un costruttore `cons` in meno. La definizione raggiunge quindi il caso `vuota`.

1.6.3 6.3 Formulare una proprietà di correttezza

Una proprietà dell'algoritmo è che rovesciare due volte restituisca la lista iniziale:

```
lecture07_rovescia_rovescia {α : Type} (xs : Lista α) :
  rovescia (rovescia xs) = xs
```

L'enunciato non riguarda soltanto `listaEsempio`. Deve valere per ogni tipo `α` e per ogni lista `xs : Lista α`. Si tratta di una proprietà importante di correttezza, anche se non costituisce da sola una specifica completa di ogni aspetto del rovesciamento.

Per dimostrare questa proprietà useremo alcuni risultati su `concatena` e sul comportamento di `rovescia` rispetto alla concatenazione. Li dimostriamo prima della proprietà finale e osserviamo, nei goal intermedi, quando ciascuno diventa necessario.

1.6.4 6.4 Proprietà della concatenazione

Concatenare la lista vuota a destra non modifica una lista. Poiché `concatena` analizza il primo argomento, dimostriamo il risultato per induzione su `xs`.

```
theorem lecture07_concatena_vuota_destra
    α{ : Type} (xs : Lista α) :
    concatena xs .vuota = xs := by
  induction xs with
  | vuota =>
      rfl
  | cons x xs ih =>
      simp [concatena, ih]
```

Nel caso `vuota`, `concatena vuota vuota` si riduce direttamente a `vuota`. Nel caso `cons x xs`, la definizione conserva `x` e lascia da dimostrare il risultato sulla coda `xs`. L'assunzione induttiva risolve precisamente quella sottoespressione.

Questa uguaglianza elimina una concatenazione con la lista vuota e produce una forma più semplice. Possiamo quindi aggiungerla al simp-set locale:

```
attribute [local simp] lecture07_concatena_vuota_destra
```

Ci serve inoltre l'associatività della concatenazione. Facciamo ancora induzione su `xs`, perché è il primo argomento della concatenazione esterna a sinistra ed è anche l'argomento che guida la definizione. Nel passo `cons`, entrambe le espressioni conservano la stessa testa `x`. L'assunzione induttiva allinea le concatenazioni delle code.

```
theorem lecture07_concatena_assoc
    α{ : Type} (xs ys zs : Lista α) :
    concatena (concatena xs ys) zs =
```

```
    concatena xs (concatena ys zs) := by
  induction xs with
  | vuota =>
      rfl
  | cons x xs ih =>
      simp [concatena, ih]
```

Non aggiungiamo invece l'associatività al simp-set. Questa uguaglianza cambia il raggruppamento, ma non indica in generale una forma più semplice. La nominiamo soltanto nei comandi che ne hanno bisogno.

1.6.5 6.5 Rovesciare una concatenazione

Quando rovesciamo due liste concatenate, le due parti compaiono in ordine inverso. Questo lemma servirà nella dimostrazione della proprietà finale.

```
attribute [local simp] lecture07_concatena_vuota_destra

theorem lecture07_rovescia_concatena
    α{ : Type} (xs ys : Lista α) :
    rovescia (concatena xs ys) =
      concatena (rovescia ys) (rovescia xs) := by
  induction xs with
  | vuota =>
      simp [concatena, rovescia]
  | cons x xs ih =>
      simp [concatena, rovescia, ih, lecture07_concatena_assoc]
```

Nel caso `vuota`, `simp` usa automaticamente il lemma locale sulla concatenazione con la lista vuota. Nel caso `cons`, l'assunzione induttiva tratta la coda e `lecture07_concatena_assoc` dispone nello stesso modo le concatenazioni sui due lati.

Anche il lemma sul rovesciamento di una concatenazione descrive una forma che vogliamo usare automaticamente nella proprietà finale. Lo aggiungiamo quindi al simp-set locale:

```
attribute [local simp] lecture07_rovescia_concatena
```

1.6.6 6.6 Rovesciare due volte

Possiamo ora dimostrare la proprietà iniziale. L'induzione segue i due costruttori di `Lista α`. Nel caso `cons`, il lemma precedente descrive il rovesciamento della concatenazione prodotta dall'algoritmo e l'assunzione induttiva semplifica il doppio rovesciamento della coda.

```
theorem lecture07_rovescia_rovescia
    α{ : Type} (xs : Lista α) :
    rovescia (rovescia xs) = xs := by
```

```
  induction xs with
  | vuota =>
      simp [rovescia]
  | cons x xs ih =>
      simp [rovescia, ih, concatena,
        lecture07_concatena_vuota_destra,
        lecture07_rovescia_concatena]
```

Questo esempio collega tre livelli. I costruttori `vuota` e `cons` descrivono la struttura dei dati. `concatena` e `rovescia` sono programmi ricorsivi su quella struttura. I teoremi esprimono proprietà attese e il kernel controlla che le dimostrazioni coprano ogni input.

La proprietà dimostrata è significativa, ma non è una specifica completa di `rovescia`. Anche una funzione che restituisse sempre la lista ricevuta soddisferebbe l'equazione del doppio rovesciamento. Per descrivere più precisamente il comportamento dell'algoritmo possiamo aggiungere altre proprietà, come il lemma che collega `rovescia` e `concatena`, oppure una proprietà che stabilisca come cambia la posizione degli elementi.

1.7 Strategia operativa

1. Individuiamo i costruttori del tipo. 2. Leggiamo quale argomento è analizzato dalla definizione. 3. Proviamo se il goal si riduce con `rfl`. 4. Usiamo con `rw` le uguaglianze già dimostrate. 5. Se resta una variabile nell'argomento analizzato, scegliamo quella variabile per l'induzione. 6. Automatizziamo con `simp only` o `simp`. 7. Nella verifica di un programma, formuliamo prima la proprietà attesa e introduciamo soltanto i lemmi ausiliari richiesti dalla dimostrazione.

1.8 Fonti

- Lean developers, *Theorem Proving in Lean 4*, «Inductive Types»: https://lean-lang.org/theorem_proving_in_lean4/Inductive-Types/.
- Lean developers, *Theorem Proving in Lean 4*, «Induction and Recursion»: https://lean-lang.org/theorem_proving_in_lean4/Induction-and-Recursion/.
- Lean developers, *Functional Programming in Lean*, «Datatypes and Patterns»: https://lean-lang.org/functional_programming_in_lean/Getting-to-Know-Lean/Datatypes-and-Patterns/.
- Lean developers, *The Lean Language Reference*, «Recursive Definitions»: https://lean-lang.org/doc/reference/latest/Definitions/Recursive-Definitions/.
- Lean community, *Natural Number Game*: https://adam.math.hhu.de/.
- Vrije Universiteit Amsterdam, *Logical Verification*: https://research.vu.nl/en/courses/logical-verification-5/.